

CAHIER DES CHARGES

Extracteur B2B — Plateforme d'extraction et de centralisation
des entreprises en Belgique

Champ	Détail
Intitulé	Extracteur B2B — Module Social Prospection v2
Type	Application Web Full Stack + Microservice IA
Cible géographique	Marché belge (codes postaux belges)
Architecture	Next.js · Express.js · Flask (IA) · MongoDB
Approche IA	Agent LangGraph + RAG + Scraping OSM
LLM	Groq — Llama 3.3 70B
Version	1.0 — Cahier des charges fonctionnel et technique

1. Contexte et présentation du projet

Les entreprises B2B ont besoin de bases de données fiables pour la **prospection commerciale**, les **études de marché** et la **veille concurrentielle**. Le projet vise à **centraliser** ces informations au sein d'une seule plateforme, en exploitant des sources ouvertes (OpenStreetMap / Overpass, Nominatim) et un pipeline IA (agent conversationnel + RAG) pour enrichir automatiquement les fiches entreprises identifiées par code postal et domaine d'activité (boulangerie, restaurants, bureaux, commerces, etc.).

Cette v2 étend le module **Social Prospection** existant en ajoutant un **nouveau canal de sourcing** (OSM Belgique) piloté par un agent en langage naturel, avec enrichissement RAG pour combler les données manquantes (téléphone, e-mail, site web) fréquemment absentes des POI OSM.

2. Objectifs

- Développer une application web de recherche et de centralisation des entreprises belges.
- Permettre la recherche par **code postal**, **ville/province**, **secteur** et **rayon géographique**.
- Automatiser la collecte via **Overpass API + Nominatim** et l'enrichissement via **RAG**.
- Offrir un **agent conversationnel** pour piloter les actions en langage naturel.
- Générer des **bilans de prospection** structurés (secteur, score, argumentaire).
- Exporter les données aux formats **Excel, CSV, JSON**.

3. Fonctionnalités principales

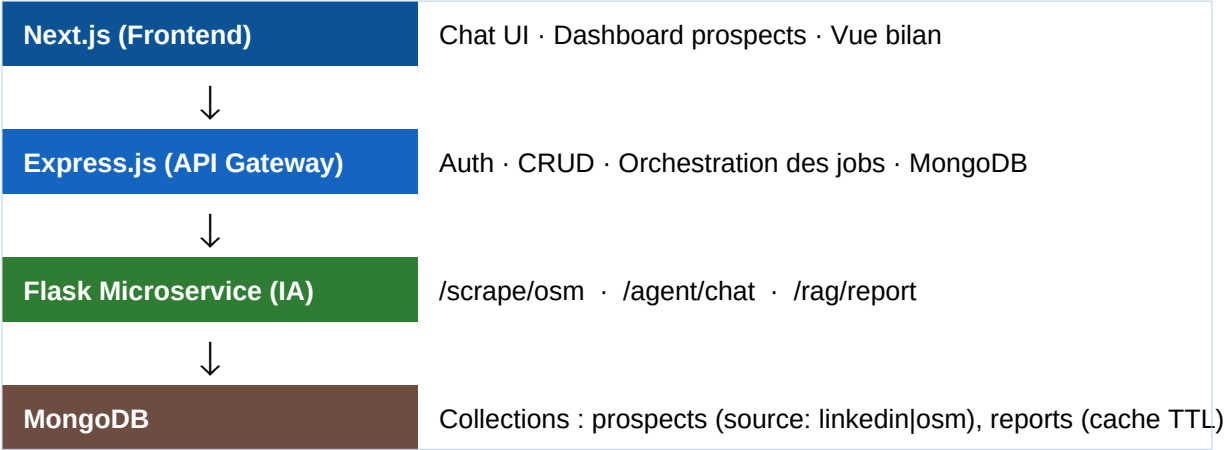
Module	Description
Recherche d'entreprises	Par secteur d'activité, ville, province, code postal, ou rayon géographique autour d'un point.
Gestion des fiches	Nom, catégorie, adresse, téléphone, e-mail, site web, note/avis, coordonnées GPS, source.
Tableau de bord	Nombre d'entreprises, répartition par secteur et région, statistiques temps réel, KPIs prospection.
Recherche avancée	Filtres par catégorie, ville/province, présence e-mail/téléphone, note minimale, source (OSM/LinkedIn).
Export de données	Export Excel (XLSX), CSV et JSON avec sélection de colonnes et filtres appliqués.
Gestion utilisateurs	Authentification JWT, rôles (admin, commercial, viewer), gestion fine des permissions.
Agent conversationnel	Chat IA pour lancer scraping, recherches, bilans en langage naturel (FR).
Bilan RAG	Génération automatique d'une fiche bilan enrichie (secteur, présence digitale, score, argumentaire).

4. Architecture globale

L'architecture suit un pattern **3-tiers + microservice IA**, avec séparation claire des responsabilités : **Agent** (orchestration) ≠ **RAG** (connaissance) ≠ **Scraper** (collecte).

Couche	Langage(s)	Frameworks / Outils
Frontend	TypeScript / JavaScript	Next.js, React, Tailwind CSS, shadcn/ui
Backend API	JavaScript (Node.js)	Express.js, Mongoose, JWT, Zod
Base de données	—	MongoDB (NoSQL, schéma unifié Prospect)
Microservice IA	Python 3.11+	Flask, Pydantic, Uvicorn/Gunicorn
Scraping	Python	requests, Overpass API, Nominatim
Agent / Orchestration	Python	LangGraph, LangChain
LLM Inference	Python (SDK)	Groq API — Llama 3.3 70B
RAG / Embeddings	Python	FAISS (vector store), Sentence Transformers
Communication	—	REST (JSON / HTTP) entre Express ↔ Flask
DevOps	—	Docker, Docker Compose, GitHub Actions

Schéma du pipeline :



5. Les 3 briques fonctionnelles

Brique	Rôle	Langage	Techno / Librairies
1. Scraper OSM Belgique	Collecte de prospects par code postal + catégorie	Python	Flask, requests, Overpass API, Nominatim
2. Agent conversationnel	Pilotage des actions en langage naturel	Python	LangGraph, LangChain, Groq SDK (Llama 3.3 70B)
3. RAG enrichissement	Recherche complémentaire + génération de bilan	Python	LangChain, FAISS, Sentence Transformers, Groq

5.1 Brique 1 — Scraping OSM (Python)

Flux :

- Code postal belge → **bbox** via Nominatim (User-Agent obligatoire).
- Bbox + catégorie (office, shop, amenity...) → requête **Overpass QL**.
- Parsing + normalisation (nom, tel, site, adresse, lat/lon).
- Stockage MongoDB avec source: "osm".

Contraintes :

- Rate limit **Nominatim** : **1 req/sec max** (sinon ban IP).
- **Cache des bbox** par code postal pour éviter le re-géocodage.
- Beaucoup de POI sans téléphone/site → comblé par la **brique RAG**.

5.2 Brique 2 — Agent conversationnel (Python)

Pattern **LangGraph + Groq**, inspiré du Job Application Agent.

Node	Rôle
classify_intent	Détecte ce que veut l'utilisateur (scrape, bilan, info...).
scrape_osm	Déclenche le scraping Overpass + Nominatim.
rag_search	Lance la recherche enrichie sur une entreprise.
generate_report	Produit le bilan structuré (JSON).
clarify	Demande des informations manquantes (ex: code postal absent).

Exemples de commandes :

- « Trouve-moi des bureaux à 1000 Bruxelles »
- « Fais un bilan sur [Entreprise X] »
- « Liste les restaurants à Liège avec un site web »

5.3 Brique 3 — RAG + Bilan (Python)

Flux :

- Entreprise identifiée (scraping ou saisie manuelle).
- Collecte complémentaire (site web, recherche web) si données OSM incomplètes.
- **Indexation** : chunking + embeddings (Sentence Transformers) → FAISS.
- **Génération du bilan** structuré (JSON) via Groq LLM : secteur, présence digitale, contact, **score de prospection**, argumentaire suggéré.

Cache recommandé : bilan stocké avec **TTL ~30 jours** pour éviter coûts et latence répétés.

6. Endpoints REST exposés

Méthode	Endpoint	Service	Description
POST	/api/auth/login	Express	Authentification JWT.
GET	/api/prospects	Express	Liste paginée avec filtres (source, ville, score...).
POST	/api/prospects/export	Express	Export Excel / CSV / JSON.
GET	/api/dashboard/stats	Express	Statistiques (par secteur, région, source).
POST	/scrape/osm	Flask	Lance le scraping OSM (code postal + catégorie).
POST	/agent/chat	Flask	Routeur LangGraph (conversation utilisateur).
POST	/rag/report	Flask	Génère un bilan RAG pour une entreprise.

7. Modèle de données — MongoDB

Schéma **Prospect** unifié (champ source distinguant LinkedIn simulé / OSM) :

Champ	Type	Description
_id	ObjectId	Identifiant MongoDB.
name	String	Nom de l'entreprise.
category	String	Secteur d'activité (boulangerie, office, restaurant...).
address	Object	{ street, city, postcode, province, country }.
location	GeoJSON Point	{ type: 'Point', coordinates: [lon, lat] } — index 2dsphere.
phone / email / website	String	Coordonnées (souvent nulles côté OSM).
rating	Number	Note moyenne (si disponible).
source	Enum	"linkedin" "osm" — origine de la donnée.
score	Number	Score de prospection calculé par le RAG.
createdAt / updatedAt	Date	Timestamps automatiques.

Collection **reports** (cache des bilans RAG) :

Champ	Type	Description
prospectId	ObjectId	Référence vers le prospect.
payload	JSON	Bilan structuré généré par le LLM.
generatedAt	Date	Date de génération.
expiresAt	Date	TTL ~30 jours (index TTL MongoDB).

8. Sécurité & contraintes non fonctionnelles

- **Authentification** : JWT, hash bcrypt, refresh tokens.
- **Autorisations** : rôles (admin, commercial, viewer) + middleware RBAC.
- **Rate limiting** : Express (express-rate-limit) + respect strict Nominatim (1 req/sec).
- **RGPD** : données B2B uniquement, base légale intérêt légitime, registre des traitements.
- **Validation** : Zod côté Express, Pydantic côté Flask.
- **Logs & monitoring** : Winston (Node), structlog (Python), dashboard Grafana optionnel.
- **Performance** : pagination, index MongoDB (text + 2dsphere), cache Redis envisageable.
- **Robustesse scraping** : User-Agent custom obligatoire, gestion 429/timeouts, retries exponentiels.

9. Livrables

- Application web fonctionnelle (Next.js + Express + Flask + MongoDB).
- Base de données structurée et peuplée d'un jeu de test.
- API REST documentée (Swagger / OpenAPI).
- Conteneurisation Docker + docker-compose.yml.
- Documentation technique (architecture, schémas, README).
- Documentation utilisateur (guide chat agent + dashboard).
- Scripts de déploiement et de seed.
- Rapport PFE + soutenance.

10. Planning prévisionnel (12 semaines)

Phase	Durée	Livable
Étude & conception	2 sem.	Cahier des charges, maquettes, schéma BD, archi.
Setup & socle technique	1 sem.	Repos Git, Docker, CI/CD, scaffolding Next/Express/Flask.
Brique 1 — Scraper OSM	2 sem.	Endpoint /scrape/osm + cache géocodage.
Brique 2 — Agent LangGraph	2 sem.	Endpoint /agent/chat + nodes principaux.
Brique 3 — RAG + Bilan	2 sem.	Endpoint /rag/report + cache TTL Mongo.
Frontend & intégration	2 sem.	Dashboard, chat UI, exports, auth.
Tests, doc & déploiement	1 sem.	Tests, documentation, soutenance.

11. Compétences développées

- Développement **Full Stack** (Next.js, Express, Flask).

- Conception de **bases de données NoSQL** et requêtes géospatiales.
- Conception et consommation d'**API REST**.
- **Intégration IA** : LLM, RAG, agents (LangGraph / LangChain).
- **Web scraping** responsable (Overpass, Nominatim, rate limiting).
- Gestion et traitement de **données massives**.
- **Déploiement** (Docker, CI/CD, monitoring).

Document généré dans le cadre du PFE — Extracteur B2B Belgique · v1.0